

INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UNIDAD CULHUACÁN

Reporte de proyecto

Ecosistema multiagente para la generación y validación de circuitos analógicos CA/CD a partir de lenguaje natural

PRESENTAN:

ANTONIO DE JESUS TORRES MARTÍNEZ

DIEGO JESUS GUTIERREZ DELGADO

ASESOR:

M. en C. RICARDO ISRAEL CALZADA SALAS



Ciudad de México 13 de octubre de 2026

Índice

Introducción **1**

Planteamiento del Problema . . . 1
Objetivo General . . . 2
Objetivos Específicos . . . 2
Estado del Arte . . . 3
AnalogCoder . . . 3
AnalogAgent . . . 3
CircuitLM . . . 4
Justificación . . . 4
Límites y Alcances . . . 5

1. Marco Teórico **6**

1.1. Modelos de lenguaje grande . . . 6
1.2. Sistemas multiagente y orquestación . . . 7
1.2.1. LangGraph . . . 7
1.2.2. LangFuse . . . 8
1.3. Entrenamiento de Agentes para el diseño de circuitos analógicos . . . 9
1.3.1. Reglas de Prompts . . . 9
1.3.2. Generacion Aumentada por Recuperación (RAG) . . . 9
1.3.3. Spice (Simulation Program with Integrated Circuit Emphasis) . . . 11
1.3.4. PySpice . . . 11

2. Diseño y desarrollo del proyecto **12**

2.1. Diseño Conceptual . . . 12
2.2. Descomposición funcional y casos de uso . . . 13
2.2.1. Vista funcional . . . 13
2.2.2. Modelo de casos de uso . . . 14
2.3. Arquitectura del sistema (modelo C4) . . . 15
2.3.1. Nivel 1 — Contexto . . . 15
2.3.2. Nivel 2 — Contenedores . . . 16
2.3.3. Nivel 3 — Componentes . . . 17
2.4. Comportamiento dinámico: flujo de generación . . . 18

2.5. Síntesis del diseño 21
2.6. Desarrollo 21
2.7. Código fuente del proyecto 22
2.8. Pantallas del prototipo funcional 23

3. Pruebas y Análisis de Resultados **26**

4. Conclusiones y Trabajo Futuro **27**

Referencias **28**

A. Diagramas de casos de uso **30**

Índice de figuras

2.1. Diseño conceptual del sistema. 13
2.2. Diagrama funcional del sistema. 14
2.3. Diagrama de casos de uso: flujo de simulación y validación. 15
2.4. C4 Nivel 1: Diagrama de contexto. 16
2.5. C4 Nivel 2: Diagrama de contenedores. 17
2.6. C4 Nivel 3: Diagrama de componentes. 18
2.7. Diagrama de secuencia del flujo de generación. 20
2.8. Diagrama funcional del sistema. 22
2.9. Pantalla de inicio de sesión del prototipo, con la ruta de solución del pipeline de generación. 24
2.10. Espacio de trabajo autenticado: solicitud en lenguaje natural, resumen operativo, actividad reciente y archivos generados. 25

A.1. Diagrama de casos de uso: acceso y gestión de sesiones. 30
A.2. Diagrama de casos de uso: generación de circuito. 31
A.3. Diagrama de casos de uso: decisión del agente curador. 31

Introducción

Planteamiento del Problema

Con el avance tecnológico, se le han encontrado muchas utilidades a los Modelos de Lenguaje de Gran tamaño. En este sentido la generación de circuitos analógicos mediante lenguaje natural tiene un percance, por la manera en que funcionan los LLMs, rara vez devuelven una solución satisfactoria puesto que trabajan mediante distribuciones probabilísticas y no por leyes físicas. Esta manera de funcionar resulta en defectos en la generación del circuito, sumando a que utilizar un mismo agente/LLM tiene una deficiencia en la memoria, creando ambigüedad y alucinaciones del mismo.

A diferencia del diseño digital, que se apoya en abstracciones estandarizadas y flujos de automatización maduros, el diseño de circuitos analógicos descansa en el conocimiento experto, el razonamiento heurístico y un ajuste iterativo guiado por simulación [1]. Ese ciclo —proponer un circuito, simularlo, revisar los resultados y volver a ajustar— se repite numerosas veces antes de obtener un diseño que cumpla las metas, y validar un diseño completo puede exigir largos periodos de iteración [2]. Buena parte de ese conocimiento reside en la experiencia de quien diseña y no en un procedimiento que pueda seguirse paso a paso, lo que vuelve el diseño analógico difícil tanto de automatizar como de aprender.

Los Modelos de Lenguaje Grande (LLM) abrieron la posibilidad de describir un circuito en lenguaje natural y recibir una propuesta de diseño de forma automática. Al llevarla a la práctica, sin embargo, aparecen limitaciones documentadas. Estos modelos alucinan componentes, violan restricciones físicas del circuito y producen salidas eléctricamente inconsistentes [3], por lo que sus diseños rara vez funcionan en el primer intento, ya que el modelo no comprueba por sí mismo si lo que generó cumple.

Los trabajos publicados se reparten entre dos líneas. La primera diseña a nivel de transistor, atado a un proceso de fabricación específico descrito por su *Process Design Kit* (PDK), y emplea la simulación SPICE como realimentación durante el diseño; es la línea más desarrollada [1], [4], [5]. La segunda trabaja a nivel de placa, con componentes comerciales, y verifica el resultado mediante reglas eléctricas [2], [3], en parte porque no todo componente comercial cuenta con un modelo SPICE utilizable, lo que impide simular cualquier diseño de forma uniforme. La simulación SPICE a nivel de placa ha comenzado a aparecer, pero en forma de banco de prueba que compara el diseño generado contra un circuito de referencia construido por expertos [6], y no como una señal que el propio sistema utilice para ajustar el

diseño durante su ejecución. Entre los trabajos revisados no se encontró un sistema generativo de varios agentes que use la simulación SPICE como realimentación dentro del lazo de diseño, con un curador que decida el ajuste, orientado a circuitos analógicos CA/CD con componentes comerciales. Ese es el punto del espacio de diseño que este proyecto se propone ocupar.

Objetivo General

Desarrollar y evaluar un ecosistema multiagente basado en LLMs y Generación Aumentada por Recuperación (RAG) para la síntesis, dimensionamiento y verificación automatizada de circuitos analógicos CA/CD con componentes comerciales (COTS) a partir de lenguaje natural, garantizando su convergencia eléctrica funcional mediante simulación en lazo cerrado con ngspice.

Objetivos Específicos

1. **Desarrollar** una interfaz web modular con autenticación y gestión de credenciales basada en Better Auth y Drizzle ORM sobre PostgreSQL/Neon, que permita al usuario parametrizar restricciones de diseño analógico en lenguaje técnico y visualizar de forma integrada el código netlist generado, la documentación funcional del circuito, el esquema resultante y los gráficos derivados de la emulación eléctrica.
2. **Entrenar e implementar** un ecosistema multiagente coordinado mediante grafos de estado en LangGraph, integrando un módulo de cálculo y un agente curador sustentado en Generación Aumentada por Recuperación (RAG) para consultar especificaciones de componentes comerciales (COTS), gobernado por un lazo de retroalimentación acotado a un presupuesto estricto de máximo 10 iteraciones de auto-corrección.
3. **Desplegar** una arquitectura de microservicios desacoplada en contenedores Docker independientes para el cliente y el servidor stateless, configurando la pasarela de llamadas hacia APIs externas de LLMs y habilitando la ejecución segura del motor ngspice mediante PySpice en un entorno de pruebas aislado (sandbox).
4. **Evaluar** el desempeño global del sistema mediante un banco de pruebas de circuitos CA/CD a nivel COTS, cuantificando la tasa de convergencia en simulación (solve rate dentro de las 10 iteraciones permitidas) y la precisión de las restricciones eléctricas frente a las metas solicitadas.

Estado del Arte

La automatización y el diseño generativo de circuitos analógicos mediante inteligencia artificial han cobrado especial relevancia en años recientes. Para situar el aporte de este trabajo, resulta indispensable analizar las metodologías propuestas en la literatura técnica reciente.

AnalogCoder

En primer lugar, **AnalogCoder** [4] introdujo la propuesta de diseñar circuitos tratándolos como código ejecutable; en lugar de pedirle al modelo de lenguaje que describa un circuito en lenguaje natural, se le instruye para que escriba un script en Python capaz de generarlo. Al convertir el diseño en un programa ejecutable se reduce significativamente la ambigüedad de la síntesis y el netlist resultante puede simularse de inmediato. El sistema funciona sin necesidad de reentrenar el modelo base y comprueba recursivamente cada propuesta con una secuencia de verificaciones por simulación SPICE, validando que el circuito simule sin nodos flotantes, que los puntos de operación en corriente continua sean correctos y que la respuesta dinámica o de frecuencia cumpla con las especificaciones. Cuando una verificación falla, la traza de error regresa al modelo para orientar el siguiente intento de corrección. No obstante, esta aproximación coordina todo el flujo con un solo agente y se limita a nivel de transistor sobre un proceso de fabricación específico.

AnalogAgent

Por su parte, **AnalogAgent** [1] reparte esta carga de trabajo mediante un esquema de colaboración multiagente. Su contribución central consiste en la gestión y destilación del conocimiento acumulado durante los fallos: cuando un intento de diseño resulta fallido, el sistema procesa el error para traducirlo en una regla heurística explícita que se incorpora a una memoria auto-evolutiva y se recupera en iteraciones futuras, reduciendo el número total de llamadas al modelo. Adicionalmente, el estudio documenta una limitación típica de los agentes monolíticos basados en modelos de lenguaje: a medida que el historial de chat crece con cada intento de corrección, el resumen de contexto tiende a diluir u omitir detalles e instrucciones técnicas críticas. Esta estrategia de acumular conocimiento para guiar los pasos subsiguientes influye directamente en la concepción del agente curador de este trabajo, aunque mantiene la limitación de operar a nivel de transistor.

CircuitLM

Finalmente, **CircuitLM** [3] comparte con este proyecto el nivel de abstracción a nivel de componentes y módulos comerciales a partir de especificaciones de lenguaje natural. Este sistema emplea una tubería multiagente dedicada a la identificación de componentes, la consulta de hojas de especificaciones y la estructuración del esquema. No obstante, se orienta a electrónica digital y embebida —microcontroladores como Arduino o ESP32— y difiere del presente trabajo en su método de validación: en lugar de integrar simulación SPICE en el lazo, verifica la topología con un motor determinista de reglas eléctricas y un modelo de lenguaje supervisor. Sus propios autores señalan la verificación con simulador en el lazo como una limitación pendiente, justamente la señal de realimentación que este proyecto incorpora.

Ubicación de este Proyecto

Junto a los desarrollos mencionados, otras propuestas complementan el panorama del área. **AnalogSAGE** [5] introduce memoria estratificada y mecanismos de autoevolución a la síntesis a nivel de transistor; **PCBSchemaGen** [2] extiende la verificación estática de reglas al ámbito de tarjetas de circuito impreso reales; y **HWE-Bench** [6] provee un entorno de evaluación comparativa que mide el rendimiento de netlists generados contrastándolos con diseños de referencia de expertos a través de simulaciones SPICE. Adicionalmente, se registran contribuciones como **ADO-LLM** [7] para la optimización bayesiana en el diseño físico y **SPICEPilot** [8] para la generación asistida de scripts de control.

Al contrastar las soluciones se observa que ninguno de los antecedentes recopilados integra simultáneamente los cuatro rasgos propuestos en este ecosistema. Mientras que los sistemas orientados a transistores explotan la simulación en lazo cerrado y el ajuste paramétrico iterativo, no contemplan el uso de componentes comerciales. Por el contrario, los enfoques comerciales evitan la simulación dinámica en el lazo de optimización o la reducen a una métrica de prueba externa. Este proyecto une la retroalimentación física por simulación de los flujos a nivel de transistor con el diseño a nivel de placa mediante componentes comerciales, implementando una arquitectura multiagente coordinada por un curador entrenado por aprendizaje por refuerzo.

Justificación

Planteamos un método diferente de utilizar múltiples agentes en un solo módulo, aplicando procesamiento en paralelo, capaces de sintetizar, dimensionar y verificar circuitos

electrónicos CA/CD de manera automatizada mediante lenguaje natural. Todo mediante una implementación en la nube, descartando la necesidad de un software propietario especializado.

La arquitectura propuesta permite que el sistema sea utilizado para un publico abierto, sin embargo, por la naturaleza del proyecto permite a estudiantes relacionados con circuitos electrónicos analógicos CA/CD tener un aditamento en sus estudios y preparación académica.

Límites y Alcances

El sistema abarca el recorrido completo desde una descripción en lenguaje natural hasta un netlist simulado en ngspice. Del lado del cliente, gestiona el registro y el inicio de sesión, los espacios de trabajo y las claves con las que el usuario envía sus solicitudes. Del lado del servidor, lee la descripción y extrae de ella los parámetros eléctricos, divide los circuitos de varias etapas en sub-bloques, calcula los valores de los componentes, genera el código PySpice y el netlist, simula en ngspice, lee las métricas, ajusta los valores de forma iterativa a través del curador y entrega al usuario el netlist final junto con sus métricas y el historial de iteraciones.

El sistema trabaja únicamente con circuitos analógicos CA/CD formados por componentes comerciales que admiten un modelo SPICE, como resistores, capacitores, inductores, fuentes, diodos, transistores y amplificadores operacionales descritos por su macromodelo. Un componente sin un modelo utilizable queda fuera de su alcance, ya que sin modelo no hay simulación y, por tanto, no hay señal de realimentación. El sistema no diseña la disposición física de la placa (PCB), no realiza síntesis a nivel de transistor para un proceso de fabricación específico (PDK) ni genera esquemáticos gráficos. Por último, los valores numéricos que aparecen en los requisitos son metas de la primera versión y pueden ajustarse durante la evaluación experimental.

Capítulo 1

Marco Teórico

El sistema propuesto se sitúa en el cruce de cuatro dominios técnicos: el diseño de circuitos analógicos, los modelos de lenguaje, la Generacion Aumentada por Recuperación y la simulación física de circuitos, articulados sobre una arquitectura de software distribuida. Este capítulo expone los conceptos que sostienen cada uno de esos dominios y, sobre todo, las relaciones entre ellos, porque la aportación del proyecto no reside en ninguno por separado, sino en su integración dentro de un lazo de generación y validación. La exposición sigue el recorrido de los datos en el sistema: desde la descripción en lenguaje natural hasta el netlist validado.

1.1. Modelos de lenguaje grande

Un modelo de lenguaje grande es un modelo estadístico, entrenado sobre grandes volúmenes de texto, que estima la probabilidad del siguiente elemento de una secuencia a partir de los anteriores. La generación procede de forma autorregresiva: el modelo produce un elemento, lo incorpora al contexto y repite el proceso, de modo que la salida se construye paso a paso [12]. Esta capacidad de generación permite que el modelo transforme una descripción en lenguaje natural en una estructura de datos o en código, que es precisamente lo que el sistema necesita en dos puntos: la extracción de los parámetros eléctricos a partir de la descripción del circuito y el cálculo de los valores de los componentes de cada sub-bloque. Internamente, el modelo procesa el texto como una secuencia de unidades llamadas tokens y opera sobre una ventana de contexto de tamaño acotado, que fija cuánta información puede considerar en una sola consulta; esta limitación reaparece más adelante como una de las razones para descomponer el circuito en sub-bloques.

Dos propiedades de estos modelos resultan pertinentes para el diseño. La primera es el aprendizaje en contexto: el modelo ajusta su respuesta a partir de los ejemplos e instrucciones incluidos en la propia consulta, sin necesidad de reentrenarlo [13]. Esto permite proporcionarle las ecuaciones de diseño de un sub-bloque como contexto y obtener los valores correspondientes. La segunda es la generación de salida estructurada, en ocasiones mediante mecanismos de llamada a funciones, que obliga al modelo a responder en un formato definido —por ejemplo,

un objeto JSON con el tipo de circuito y sus parámetros— y facilita el procesamiento posterior de la respuesta.

La limitación que motiva el problema de investigación es complementaria a estas capacidades. El modelo genera texto plausible, pero no comprueba por sí mismo lo que produce [14]. En el dominio de los circuitos, esto se traduce en componentes inexistentes, restricciones físicas violadas y salidas eléctricamente inconsistentes que rara vez funcionan al primer intento [1], [4]. El modelo no dispone de un mecanismo interno que contraste su propuesta con el comportamiento físico del circuito; produce una respuesta que parece correcta según los patrones del lenguaje, no según las leyes de la electrónica. Esta brecha entre plausibilidad lingüística y corrección física es la que justifica incorporar una verificación externa al modelo, y es el punto del que parte el diseño del sistema: la simulación aporta la comprobación que el modelo no realiza.

La generación del modelo no es determinista. En cada paso, el siguiente elemento se elige de una distribución de probabilidad, y un parámetro de temperatura regula cuánta variación se admite en esa elección. Una temperatura baja produce salidas más estables y, al fijarla, hace repetible la generación, condición necesaria para comparar ejecuciones durante la evaluación. Esta misma variabilidad sustenta la estrategia de referencia frente a la que se contrasta el sistema. El enfoque *best-of-N* consiste en pedir al modelo N propuestas independientes para la misma descripción, evaluarlas y conservar la mejor [15]. Es una forma directa de aprovechar el muestreo, pero no incorpora realimentación: cada propuesta se genera sin información sobre los fallos de las anteriores. El sistema propuesto se mide contra esta línea base a igual número de llamadas al modelo, lo que permite distinguir la mejora atribuible al lazo de ajuste de la que se obtendría por el solo hecho de muestrear más.

1.2. Sistemas multiagente y orquestación

1.2.1. LangGraph

Es un marco de trabajo (framework) de ingeniería de software orientado al desarrollo de aplicaciones agenticas y flujos multiagente cíclicos, modelándolos formalmente como Grafos Dirigidos o Máquinas de Estados Finitas (FSM). En su arquitectura, los nodos representan agentes o funciones computacionales, los bordes definen las transiciones lógicas (incluyendo bifurcaciones condicionales), y el estado (State) constituye un bus de memoria estructurado e inmutable.

LangGraph coordina de manera declarativa la interacción entre los diferentes agentes del subsistema servidor. Permite ejecutar el patrón Master-Worker en paralelo para sub-bloques

funcionales y gestionar bordes condicionales que reinyectan diagnósticos y habilitan lazos de optimización iterativos en caso de fallo.

1.2.2. LangFuse

Es una plataforma de ingeniería y observabilidad de código abierto (LLM Observability and Tracing Platform) diseñada para rastrear, auditar y monitorizar el ciclo de vida de aplicaciones basadas en modelos de lenguaje. Facilita la telemetría detallada de trazas de ejecución (traces y spans), registro de latencias, consumo de tokens/costes asociados y evaluación cualitativa/cuantitativa de salidas y llamadas a herramientas (tool calling).

Proporciona la infraestructura de auditoría y trazabilidad en tiempo de ejecución. Permite inspeccionar qué instrucciones exactas recibió cada agente, monitorear el costo computacional de las llamadas concurrentes en la fase Map-Reduce y correlacionar los errores o reintentos de diseño con las métricas de respuesta de cada modelo.

En tareas complejas de ingeniería analógica —como la síntesis, dimensionamiento y verificación de circuitos electrónicos—, el uso de un único Modelo de Lenguaje Grande (Single LLM Agent) genera un cuello de botella crítico: la atrición o saturación de la ventana de contexto (context attrition). A medida que se incrementa el número de directivas, restricciones topológicas y mensajes de depuración, el modelo tiende a alucinar nodos flotantes, interconexiones erráticas o componentes comerciales inexistentes. Un Sistema Multiagente (Multi-Agent System - MAS) resuelve esta limitación mediante el principio de descomposición funcional (divide y vencerás), desacoplando las responsabilidades en agentes autónomos con prompts pequeños, especializados y estrictamente aislados.

La orquestación centralizada no se implementa como un script secuencial rígido, sino como una Máquina de Estados Finita (FSM) estructurada como un grafo cíclico dirigido mediante LangGraph: Estado Compartido (State): Constituye el bus de datos inmutable y central donde se registran la especificación formal del circuito, las variables intermedias, las listas de componentes calculadas y el historial de iteraciones. Nodos y Bordes Condicionales: Los nodos encapsulan a los agentes especializados, mientras que los bordes condicionales evalúan en tiempo de ejecución las transiciones del flujo, gestionando lazos cerrados de optimización (feedback loops) y detenciones de seguridad ante fallas críticas.

Para resolver circuitos analógicos multietapa (como fuentes reguladas CA/CD compuestas por etapas de transformación, rectificación, filtrado y regulación lineal), el sistema implementa el patrón concurrente Master-Worker estructurado en dos fases

Nodo Maestro (Master):

Recibe la descripción desglosada y el plan del circuito desde el Orquestador. Su rol consiste en particionar el problema global en especificaciones locales independientes para

cada sub-bloque funcional y asegurar que ningún sub-bloque exceda los límites de tokens de la ventana de contexto permitida.

Sub-agentes Trabajadores (Workers):

Se instancian de manera concurrente. Cada Worker resuelve de forma exclusiva el dimensionamiento y el planteamiento analítico de su etapa asignada (cálculo de impedancias, selección de valores comerciales COTS, corrientes de polarización). Al no recibir el código ni los errores de las demás etapas, se elimina el consumo excesivo de memoria operativa del modelo.

1.3. Entrenamiento de Agentes para el diseño de circuitos analógicos

1.3.1. Reglas de Prompts

1.3.2. Generacion Aumentada por Recuperación (RAG)

En esta arquitectura, la Generación Aumentada por Recuperación (RAG) sustituye los esquemas basados en ensayo y error o aprendizaje por refuerzo puro, transformando el subsistema en un entorno guiado por conocimiento empírico y documental indexado.

El RAG se estructura a través del Módulo Curador de Conocimiento y opera en tres dimensiones fundamentales:

Recuperación Previa y Anclaje Técnico (Pre-Generation Retrieval)Cuando el usuario describe un circuito en lenguaje natural, el sistema no permite que los LLMs asuman parámetros arbitrarios. El mecanismo de RAG realiza una búsqueda vectorial y semántica sobre bases de datos técnicas locales: Catálogos de Componentes COTS (Commercial Off-The-Shelf): Hojas de datos (datasheets) y tablas paramétricas de componentes reales del mercado (diodos rectificadores, capacitores de filtrado, transistores BJT/MOSFET y reguladores integrados). Plantillas de Topologías Analógicas Validadas: Estructuras de referencia netlist probadas para el tipo de circuito solicitado (filtros pasivos, fuentes lineales, configuraciones de amplificadores).Directivas de Simulación para SPICE: Parámetros de convergencia y tolerancias (.OPTIONS, condiciones iniciales .IC).El Curador extrae estos fragmentos pertinentes y los inyecta en el prompt inicial del Master y de los Workers, restringiendo el espacio de diseño a parámetros realizables en hardware físico real.

B. Mitigación de Alucinaciones Topológicas y de SizingLos modelos de propósito general tienden a inventar valores ideales (p. ej., resistores de valores no normalizados) o directivas incompatibles con los motores SPICE. La inyección dinámica de contexto mediante

RAG actúa como un filtro (grounding): obliga al agente de cálculo a elegir exclusivamente modelos que cuenten con archivos de biblioteca (.lib / .model) reconocidos por el simulador ngspice.

Lazo Cerrado de Curación y Memoria de Diseño (Knowledge Curation Loop)La aportación diferencial del RAG en este sistema radica en su funcionamiento en lazo cerrado tras la ejecución física en ngspice:

Captura del Diagnóstico: Si ngspice reporta una divergencia numérica (fallo de Newton-Raphson, timestep too small) o una discrepancia severa respecto a la especificación esperada, el Agente Shell captura el log crudo. Destilación Curada: En lugar de reinyectar el mensaje de error textual íntegro al modelo generador, el Curador procesa el fallo, identifica la causa física subyacente y genera una directiva de ajuste o regla de diseño aprendida. Consolidación en la Base Vectorial: Esta heurística verificada se almacena en el almacén de conocimiento persistente (Self-Evolving Memory / Base Vectorial). En el reintento actual (o en futuros diseños de la misma familia de circuitos), el sistema recupera dicha directiva, impidiendo que el ecosistema caiga en ciclos repetitivos de error y acelerando la convergencia a una solución funcional.

El diseño de un circuito analógico no se resuelve con un procedimiento cerrado que, a partir de una especificación, devuelva una única respuesta correcta. El proyectista parte de un conjunto de metas eléctricas —una ganancia, una frecuencia de corte, un punto de polarización— y propone una topología y unos valores de componentes que, según las ecuaciones de diseño, deberían cumplirlas. La propuesta se verifica mediante simulación o medición, se comparan los resultados con las metas y, cuando hay desviación, se ajustan los valores y se repite el proceso. Este ciclo de proponer, simular, revisar y ajustar se recorre varias veces antes de que el circuito cumpla las metas [18].

Las ecuaciones de diseño son el vínculo entre el dominio de la especificación y el de la implementación. Para un amplificador no inversor con amplificador operacional, por ejemplo, la ganancia se relaciona con el cociente de dos resistencias; para un filtro RC de primer orden, la frecuencia de corte queda determinada por el producto de una resistencia y una capacitancia. Estas relaciones permiten calcular un valor inicial, pero no garantizan que el circuito real cumpla la meta, porque el modelo ideal ignora efectos que la simulación sí captura: la carga entre etapas, las no idealidades de los componentes y la interacción entre sub-bloques. De ahí que la simulación funcione como árbitro: es el mecanismo que comprueba si los valores calculados producen el comportamiento esperado.

El proyecto se restringe a circuitos analógicos de corriente alterna y corriente directa construidos con componentes comerciales —resistores, capacitores, inductores, fuentes, diodos, transistores y amplificadores operacionales descritos por macromodelo—, en contraste

con la línea de trabajo que diseña a nivel de transistor para un proceso de fabricación específico [19]. Esta distinción no es accesoria. El diseño a nivel de componente comercial trabaja con dispositivos cuyos modelos ya existen y cuyos valores se eligen de catálogos discretos, mientras que el diseño a nivel de transistor dimensiona dispositivos sobre un nodo tecnológico. La frontera entre ambos niveles delimita el alcance del proyecto y, como se argumenta en el estado del arte, marca el espacio que la propuesta ocupa.

1.3.3. Spice (Simulation Program with Integrated Circuit Emphasis)

Desarrollado originalmente en la Universidad de California en Berkeley, SPICE es el software estándar industrial y académico para la simulación numérica y análisis del comportamiento dinámico de circuitos electrónicos analógicos. Modela circuitos a partir de un listado de interconexiones (netlist) y resuelve numéricamente sistemas de ecuaciones algebraico-diferenciales no lineales empleando técnicas como el análisis nodal modificado y métodos iterativos de Newton-Raphson.

Constituye el entorno cuantitativo de validación física real (a través de motores como ngspice). Actúa como el criterio de verdad técnica indispensable para verificar parámetros como ganancia, margen de fase o rizo, y confirmar la ausencia de anomalías topológicas (nodos flotantes, cortocircuitos o fallos de convergencia).

1.3.4. PySpice

Es una biblioteca de abstracción e interfaz para Python que encapsula la sintaxis de simulación de circuitos electrónicos y establece un puente programático con motores de simulación SPICE (particularmente ngspice). Provee una interfaz orientada a objetos para declarar componentes, topologías y directivas de simulación (.AC, .DC, .TRAN, .OP) sin requerir el ensamblado manual de texto plano.

Es la herramienta empleada por el Agente de Escritura para traducir las estructuras de datos abstractas generadas por los agentes de cálculo a código de simulación estructurado y ejecutable. Asimismo, captura vectorialmente los resultados numéricos emitidos por el simulador para que el sistema evalúe el desempeño del circuito en bucle cerrado.

Capítulo 2

Diseño y desarrollo del proyecto

2.1. Diseño Conceptual

En el capítulo anterior se revisó el estado del arte de los sistemas multiagente basados en LLM para el diseño de circuitos analógicos, y se identificó que los trabajos reportados operan a nivel de transistor sobre un nodo tecnológico (PDK), mientras que la generación y validación de circuitos a nivel de componentes comerciales (COTS) con simulación SPICE dentro del lazo de diseño permanece como una brecha abierta. Este capítulo presenta la propuesta de solución que atiende esa brecha.

El propósito del capítulo es exponer el diseño del sistema antes de su construcción: las premisas que lo fundamentan, su descomposición funcional, su arquitectura estructural y su comportamiento en ejecución. Las decisiones de diseño se justifican a partir de la naturaleza del problema, de los principios de la ingeniería de software y de los hallazgos del estado del arte; los artefactos de análisis y diseño elaborados a lo largo del proyecto —la Especificación de Requisitos de Software (SRS), el modelo de casos de uso, el diagrama funcional, el modelo arquitectónico C4 de tres niveles y el diagrama de secuencia— registran y representan esas decisiones, de modo que cada una quede expresada en un artefacto verificable.

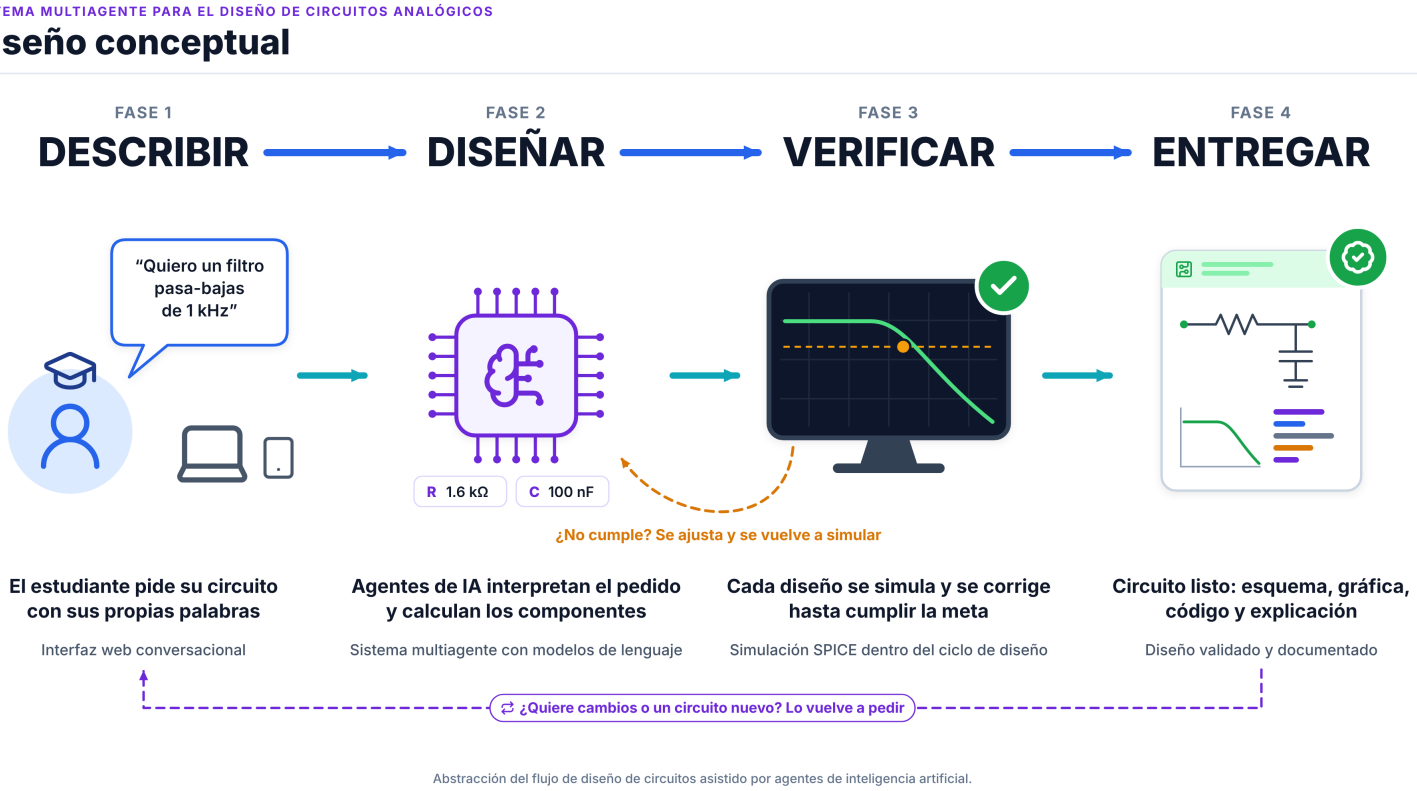


Figura 2.1. Diseño conceptual del sistema.

2.2. Descomposición funcional y casos de uso

2.2.1. Vista funcional

El sistema ofrece un conjunto acotado de funciones que se desprenden directamente del problema y quedan registradas en el SRS: dar acceso a los usuarios y gestionar sus sesiones, espacios de trabajo y API keys (cliente); leer una descripción de circuito en lenguaje natural y extraer de ella los parámetros eléctricos; dividir un circuito de varias etapas en sub-bloques; calcular los valores de los componentes de cada sub-bloque; generar el código PySpice y el netlist; simular el netlist en ngspice y leer las métricas; comparar las métricas con las metas y decidir si el circuito se acepta o se ajusta; y entregar al usuario el netlist final junto con las métricas y el historial de iteraciones.

El diagrama funcional organiza estas funciones según el subsistema responsable y el flujo de datos que las relaciona, lo que permite ver la propuesta como un encadenamiento de

transformaciones sobre la descripción del circuito, desde el lenguaje natural hasta el netlist validado.

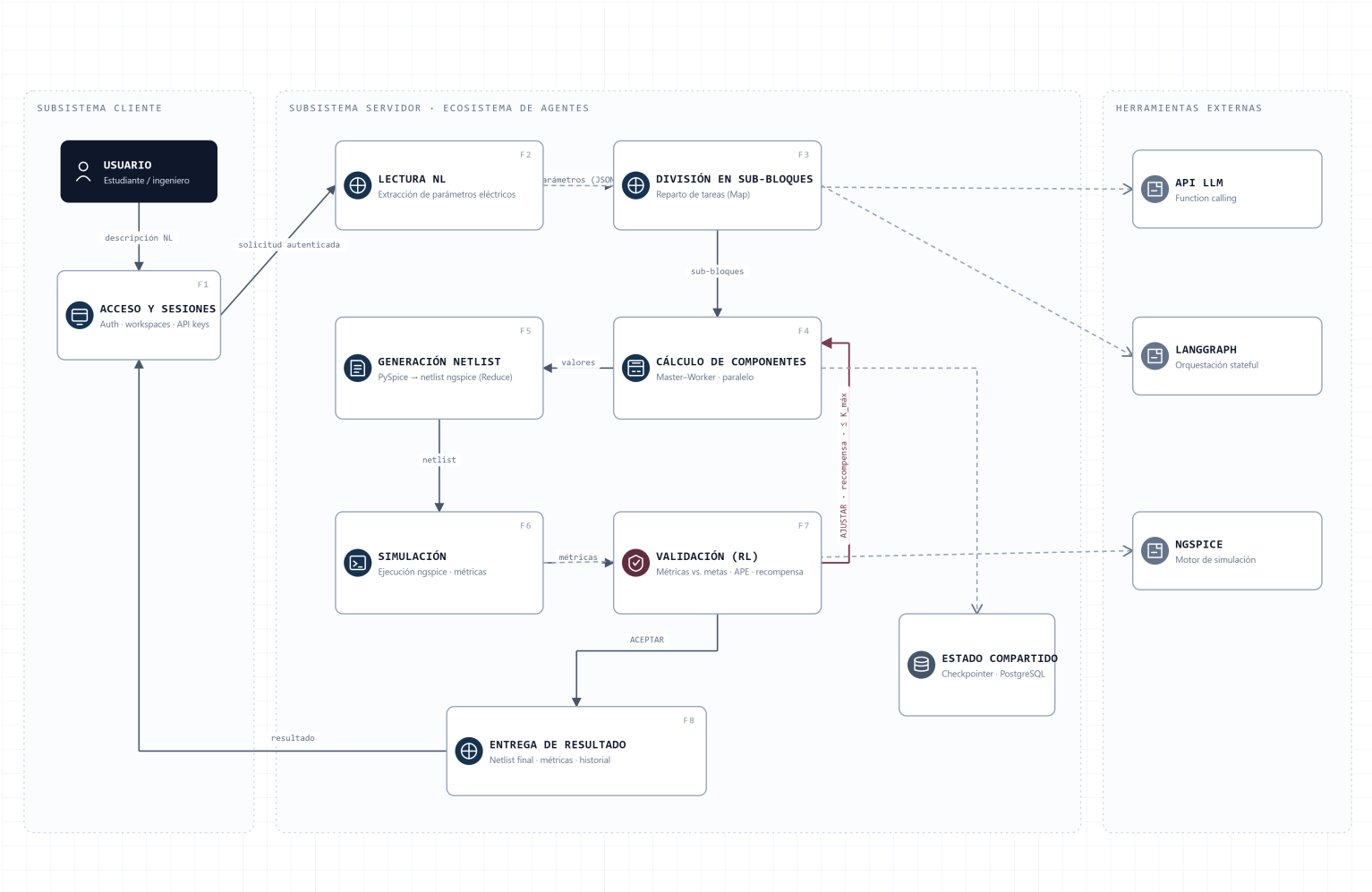


Figura 2.2. Diagrama funcional del sistema.

2.2.2. Modelo de casos de uso

Los casos de uso describen las interacciones que los actores ejecutan sobre el sistema. Se identifican dos clases de actor: el **usuario**, que interactúa con el subsistema cliente, y los **servicios externos** (la API del LLM y el motor ngspice), que el subsistema servidor consume durante la generación.

Los casos de uso principales del usuario incluyen: registrarse e iniciar sesión, gestionar espacios de trabajo, emitir y revocar API keys, y enviar una solicitud de generación de circuito junto con su descripción en lenguaje natural. El caso de uso central —*generar circuito*— desencadena el flujo del subsistema servidor, que se detalla en su comportamiento dinámico

en (sección 2.4). Los diagramas de casos de uso restantes (acceso, generación y decisión) se incluyen en el apéndice A.

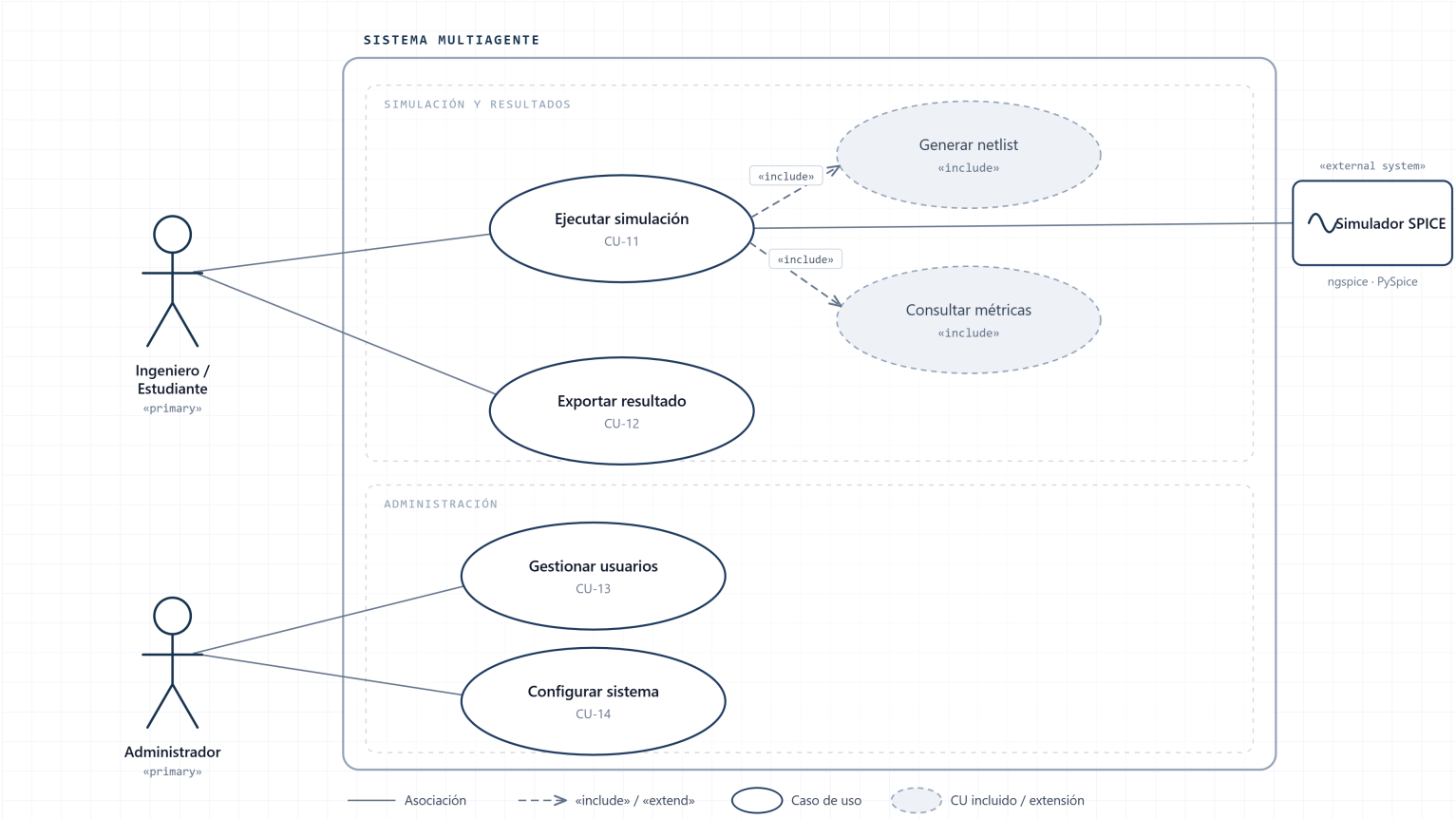


Figura 2.3. Diagrama de casos de uso: flujo de simulación y validación.

2.3. Arquitectura del sistema (modelo C4)

2.3.1. Nivel 1 — Contexto

El nivel de contexto sitúa el sistema frente a sus actores y sistemas externos. Muestra al usuario interactuando con el sistema, y al sistema consumiendo dos servicios externos: la API del LLM y el motor de simulación ngspice. Este nivel deja claro que el sistema es una unidad que media entre la intención del usuario, expresada en lenguaje natural, y dos capacidades externas: el razonamiento del LLM y la simulación física del circuito.



Figura 2.4. C4 Nivel 1: Diagrama de contexto.

2.3.2. Nivel 2 — Contenedores

El nivel de contenedores descompone el sistema en sus unidades desplegables. Hace explícita la separación cliente/servidor: el subsistema cliente (autenticación, sesiones, espacios de trabajo, API keys) y el subsistema servidor (el ecosistema de agentes), junto con la base de datos de persistencia y la frontera con los servicios externos. Este nivel justifica la premisa de despliegue independiente (sección **??**) y muestra el camino de una solicitud autenticada desde el cliente hacia el servidor.

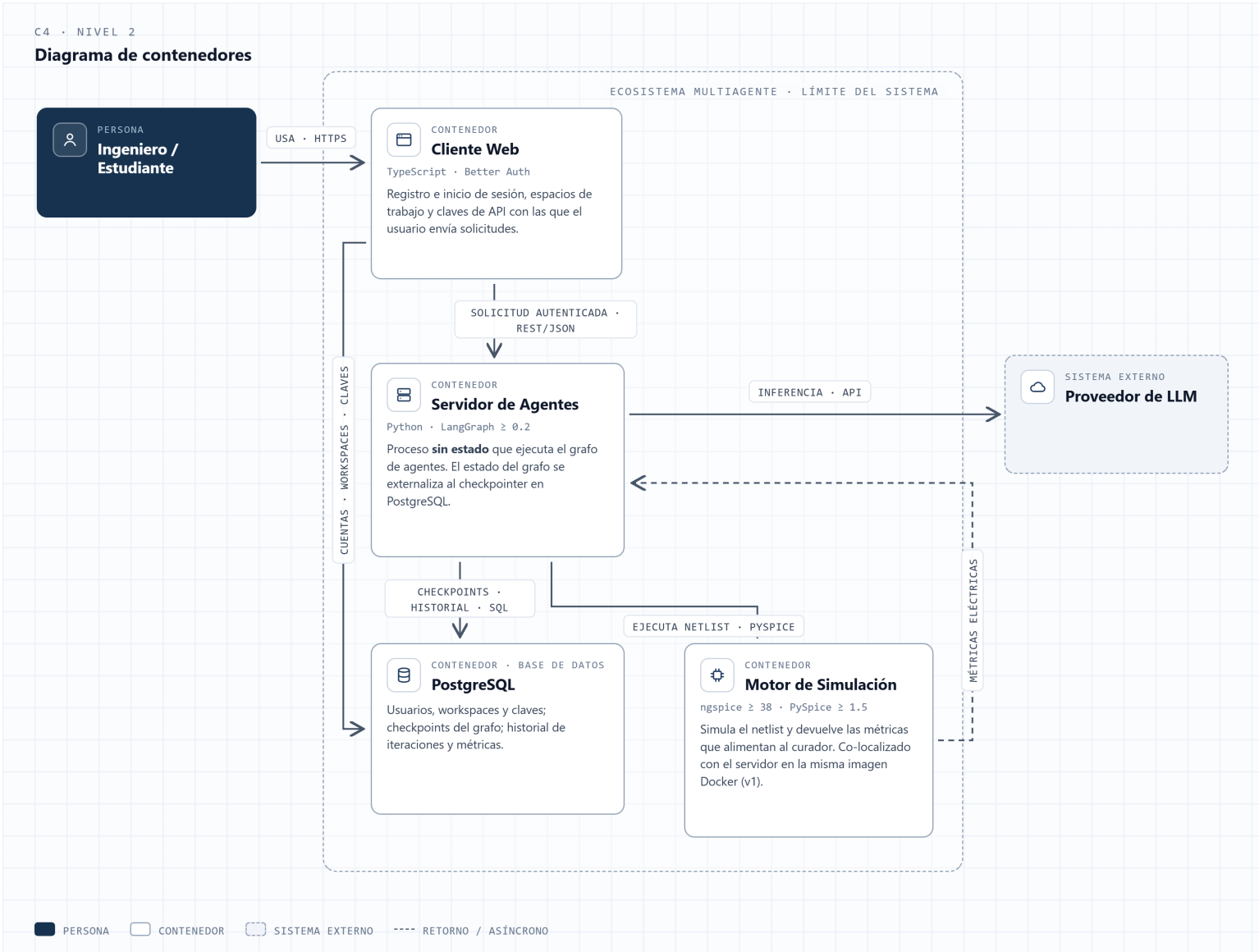


Figura 2.5. C4 Nivel 2: Diagrama de contenedores.

2.3.3. Nivel 3 — Componentes

El nivel de componentes abre el subsistema servidor y muestra el ecosistema de agentes: el agente Orquestador (entrada del servidor y coordinador del estado compartido), el agente de Cálculo (Master-Worker), el agente Curador (RL, lazo de validación iterativa), el agente de Shell (ejecución de ngspice) y el agente de Escritura (generación del netlist con PySpice), todos coordinados sobre el grafo de orquestación con persistencia en base de datos. Este nivel materializa la premisa de modularidad de agentes (sección **??**) y prepara el terreno para la descripción del comportamiento dinámico.

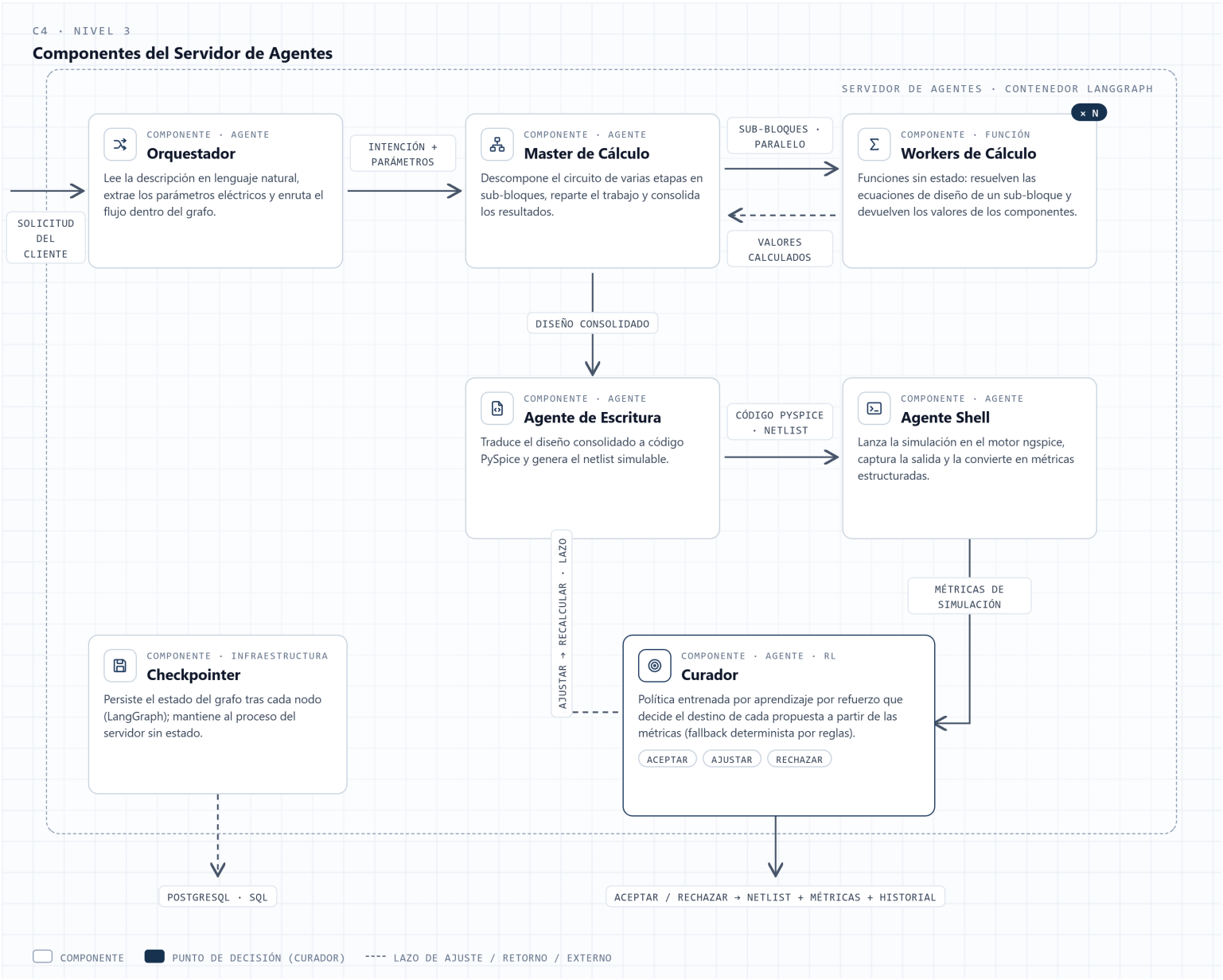


Figura 2.6. C4 Nivel 3: Diagrama de componentes.

2.4. Comportamiento dinámico: flujo de generación

Las secciones anteriores describen la estructura estática del sistema. Esta sección describe su comportamiento en ejecución: cómo colaboran los agentes en el tiempo para transformar una descripción en lenguaje natural en un netlist validado. El diagrama de secuencia integra las vistas anteriores en una línea temporal.

El flujo principal procede así: el orquestador recibe la solicitud autenticada y la descripción del circuito, usa el LLM para extraer los parámetros y la lista de sub-bloques,

y reparte los sub-bloques independientes en paralelo al agente de cálculo (Master-Worker). El agente de escritura genera el código PySpice y el netlist; el agente de shell lo simula en ngspice y lee las métricas; el agente curador compara las métricas con las metas y decide. Según su decisión, el ciclo se repite —ajustando los valores— o termina. El ciclo está acotado por un número máximo de iteraciones configurable (parámetro de guarda del lazo).

El comportamiento de persistencia se modela de forma explícita: el estado de cada iteración se conserva mediante el *checkpointer*, lo que se representa como un fragmento de referencia en el diagrama, de modo que la naturaleza con estado del sistema quede formalmente reflejada y no como una observación accesoria.

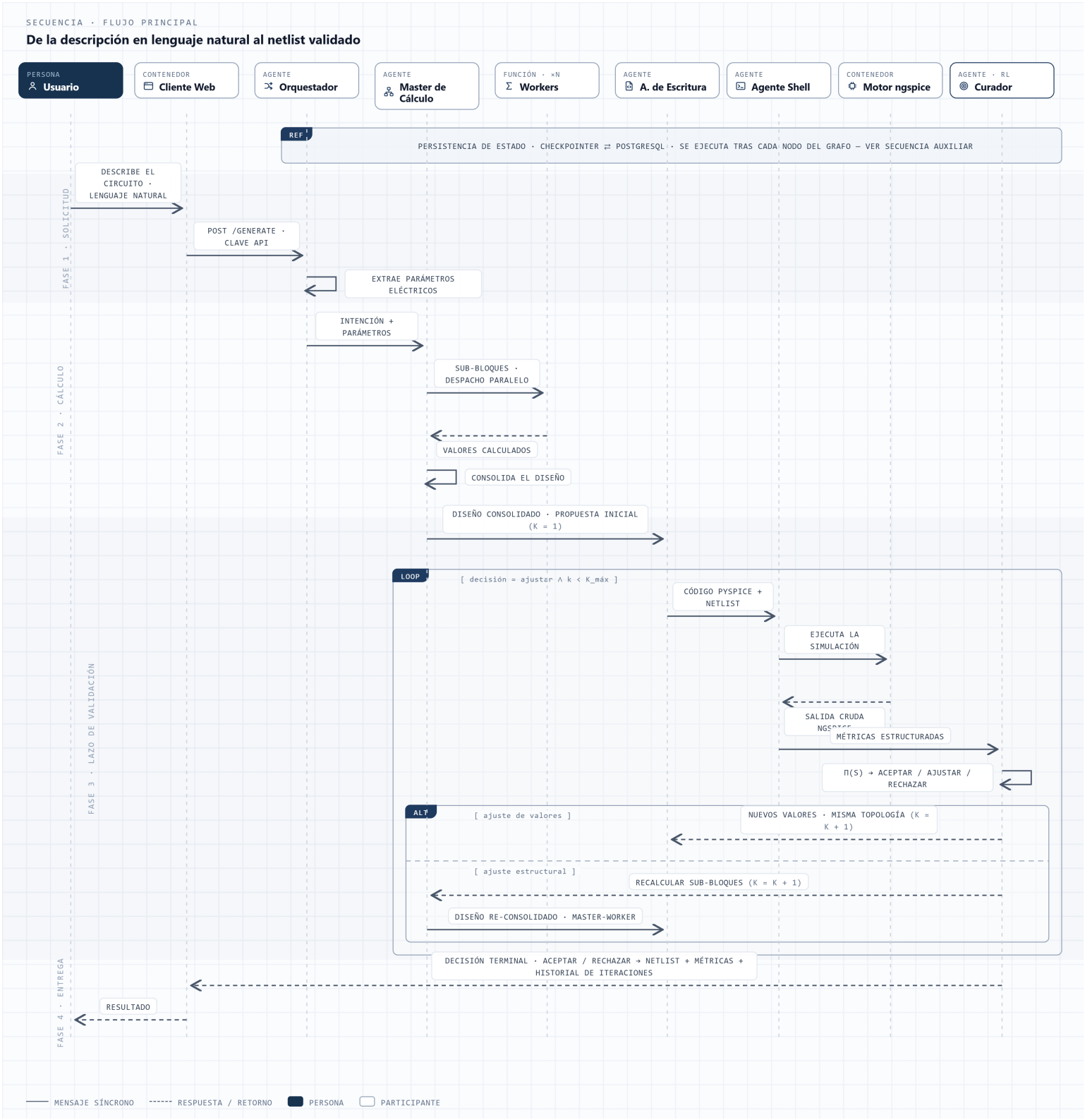


Figura 2.7. Diagrama de secuencia del flujo de generación.

2.5. Síntesis del diseño

La propuesta de solución descrita en este capítulo da forma concreta a la brecha identificada en el estado del arte: un sistema que genera y valida circuitos analógicos CA/CD a nivel de componentes comerciales, con la simulación SPICE incorporada dentro del lazo de diseño. El diseño se articula en tres planos que se sostienen entre sí. La descomposición funcional y los casos de uso (sección 2.2) establecen qué hace el sistema y quién interactúa con él. El modelo C4 en sus tres niveles (sección 2.3) establece cómo se estructura: la separación en subsistema cliente y subsistema servidor, y la organización del servidor como un ecosistema de agentes modulares —Orquestador, Cálculo, Curador, Shell y Escritura— coordinados sobre un grafo con estado y persistencia. El diagrama de secuencia (sección 2.4) establece cómo se comportan esos elementos en el tiempo, integrando el lazo de ajuste iterativo que distingue a la propuesta.

Los criterios de aceptación enunciados en (sección **??**) fijan la condición que el sistema construido deberá cumplir. Su comprobación —la medición del comportamiento del sistema frente a esos criterios sobre un conjunto de circuitos de prueba— constituye la evaluación del sistema y se desarrolla en el capítulo siguiente, que toma este diseño como objeto de medida.

2.6. Desarrollo

Anteriormente se expuso el diseño del sistema: su arquitectura en el modelo C4, su descomposición funcional y de casos de uso, su comportamiento dinámico, y los marcos de calidad (sección **??**) y de seguridad (sección **??**) que orientan la construcción. Este capítulo presenta la materialización de ese diseño en un prototipo funcional: el código fuente que lo implementa y su organización en el repositorio, las pantallas del prototipo en operación, y el análisis de seguridad de la implementación, que contrasta los controles previstos en el diseño con los efectivamente construidos.

Podemos observar un resumen de como se comporta el sistema en su totalidad.

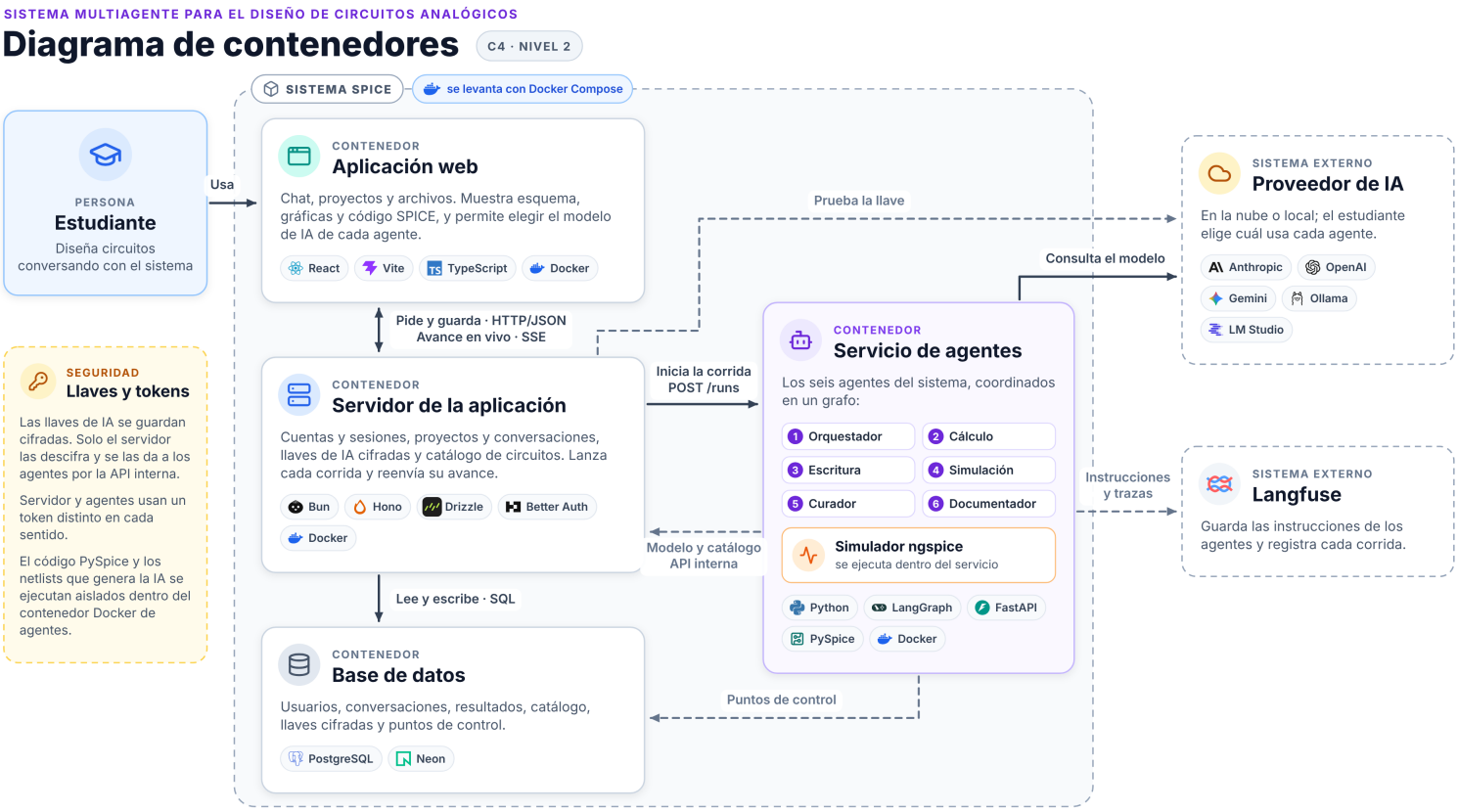


Figura 2.8. Diagrama de contenedores.

2.7. Código fuente del proyecto

El código fuente completo del sistema se encuentra publicado en un repositorio de GitHub, lo que permite su revisión, su reproducción y el seguimiento de su historial de desarrollo:

https://github.com/antoniotorresxd/ia-spice

El repositorio se organiza como un monorepositorio con tres aplicaciones independientes bajo el directorio `project/apps/`, en correspondencia directa con los contenedores del nivel 2 del modelo C4 (sección 2.3.2):

- `client/` — el subsistema cliente: una aplicación web construida con React y TypeScript sobre Vite, que concentra la autenticación, los espacios de trabajo y la interfaz desde la que el usuario envía sus solicitudes en lenguaje natural.
- `server/` — la API del sistema: un servicio construido con el framework Hono sobre el runtime Bun, con Better Auth para la autenticación, Drizzle ORM sobre PostgreSQL

(Neon) para la persistencia, y validación de esquemas con Zod. Expone, además de las rutas de autenticación, el catálogo de configuraciones de LLM que consume el subsistema de agentes.

- `agents/` — el ecosistema de agentes: un proyecto en Python 3.12 que implementa, sobre LangGraph, el grafo de orquestación descrito en el capítulo anterior —orquestador, cálculo (Master-Worker), síntesis (escritura y shell) y curador— con PySpice para la generación de netlists y el motor ngspice para la simulación.

Cada aplicación gestiona sus propias dependencias con su gestor correspondiente (`npm`, `bun` y `uv`, respectivamente) y cuenta con su propia suite de pruebas automatizadas, cuyos resultados se documentan en el capítulo de pruebas. El historial de *commits* del repositorio registra la evolución incremental del desarrollo, con mensajes bajo la convención *conventional commits* que distinguen funcionalidades, correcciones, pruebas y documentación.

2.8. Pantallas del prototipo funcional

Las siguientes capturas muestran el prototipo funcional en operación y evidencian que el subsistema cliente materializa los casos de uso de acceso y de solicitud de generación descritos en la sección 2.2.2.

La pantalla de inicio de sesión (figura 2.9) es la puerta de entrada al sistema. En su panel derecho ofrece el formulario de autenticación por correo y contraseña —delegada en Better Auth, conforme al diseño de seguridad— junto con las opciones de registro de cuenta nueva. El panel izquierdo comunica la propuesta del sistema y presenta la *ruta de solución*: el diagrama de estados del pipeline de generación (entrada, orquestador, cálculo, síntesis, curador y los estados terminales de aceptación y rechazo), de modo que el usuario conoce, desde el acceso, el proceso de validación iterativa que seguirá su solicitud.

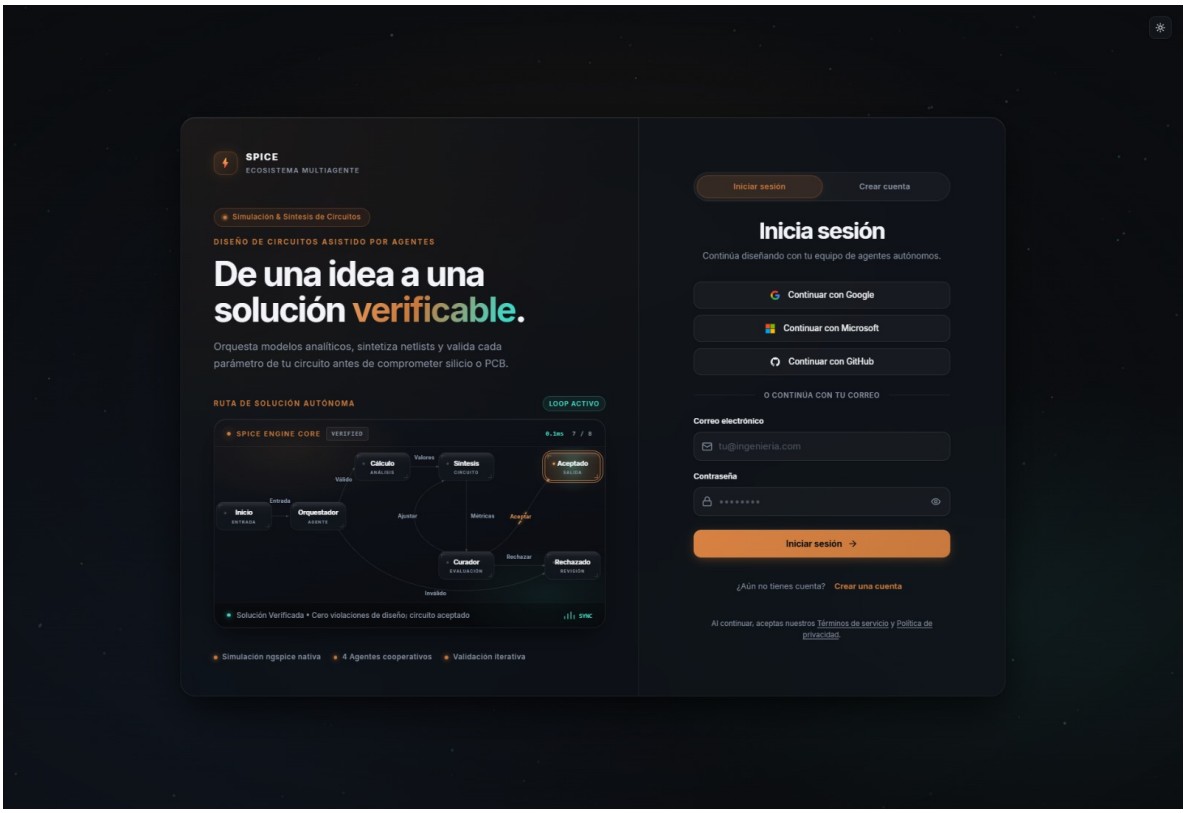


Figura 2.9. Pantalla de inicio de sesión del prototipo, con la ruta de solución del pipeline de generación.

Tras autenticarse, el usuario accede a su espacio de trabajo personal (figura 2.10). La pantalla de inicio concentra el caso de uso central del sistema: el campo de solicitud en lenguaje natural (“Describe qué quieres diseñar”), desde el cual se envía la descripción del circuito que desencadena el flujo de generación. La pantalla ofrece además un resumen operativo del consumo del periodo —tokens utilizados, costo estimado, tasa de solicitudes exitosas y tiempo procesado—, la actividad reciente con el estado de cada solicitud (completada, en proceso o que requiere atención), los proyectos que agrupan las conversaciones del usuario, y los archivos producidos por las generaciones (netlists `.cir`, resultados `.csv` y esquemáticos `.svg`). El asistente conversacional, visible en la esquina inferior, acompaña la interpretación de resultados y el ajuste de restricciones.

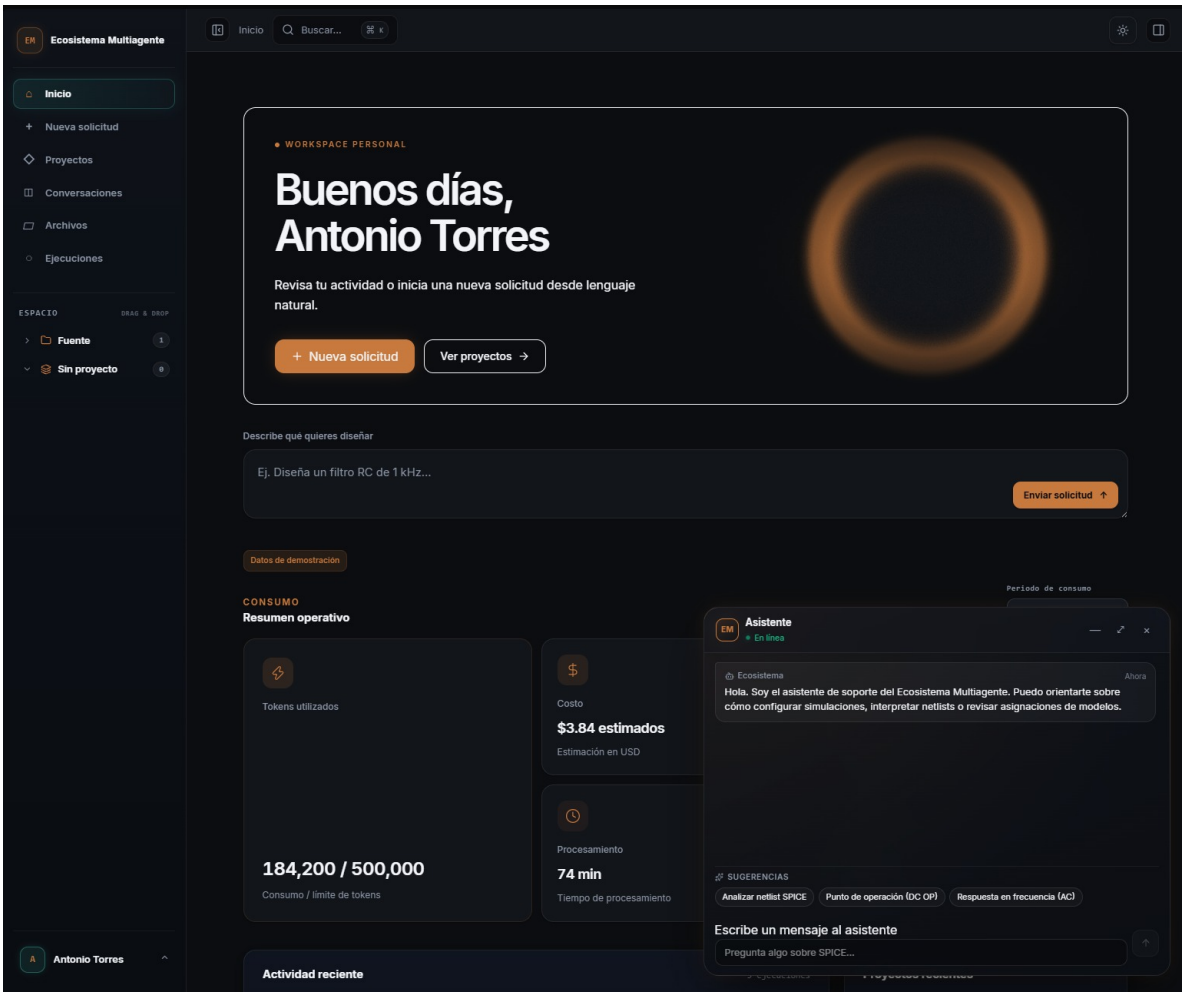


Figura 2.10. Espacio de trabajo autenticado: solicitud en lenguaje natural, resumen operativo, actividad reciente y archivos generados.

Posteriormente se diseñó un pequeño tutorial que le indica al usuario como interactuar con el sistema, que funcion cumple cada módulo y que información se espera de cada uno. El tutorial se encuentra disponible en la sección de ayuda del sistema (figura **??**).

Capítulo 3

Pruebas y Análisis de Resultados

Capítulo 4

Conclusiones y Trabajo Futuro

Referencias

[1] Z. Bao et al. «AnalogAgent: Self-Improving Analog Circuit Design Automation with LLM Agents.» arXiv: 2603.23910 [cs.AI]. dirección: https://arxiv.org/abs/2603.23910

[2] H. Zou, P. Han, E. Nazerian y A. Q. Huang. «PCBSchemaGen: Constraint-Guided Schematic Design via LLM for Printed Circuit Boards.» arXiv: 2602.00510 [cs.SE]. dirección: https://arxiv.org/abs/2602.00510

[3] K. S. Al Hasan, S. R. Raiyan, H. M. Alvee y W. Sadik. «CircuitLM: A Multi-Agent LLM-Aided Design Framework for Generating Circuit Schematics from Natural Language Prompts.» arXiv: 2601.04505 [cs.SE]. dirección: https://arxiv.org/abs/2601.04505

[4] Y. Lai et al., «AnalogCoder: Analog Circuit Design via Training-Free Code Generation,» en *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence (AAAI-25)*, 2025.

[5] Z. Wang, J. Gao, W. Fu, X. Guo y X. Zhang. «AnalogSAGE: Self-evolving Analog Design Multi-Agents with Stratified Memory and Grounded Experience.» arXiv: 2512.22435 [cs.AR]. dirección: https://arxiv.org/abs/2512.22435

[6] W. Qiu, Y. Xiao y R. Pan. «HWE-Bench: Can Language Models Perform Board-level Schematic Designs?» arXiv: 2603.18102 [cs.SE]. dirección: https://arxiv.org/abs/2603.18102

[7] Y. Yin, Y. Wang, B. Xu y P. Li. «ADO-LLM: Analog Design Bayesian Optimization with In-Context Learning of Large Language Models.» arXiv: 2406.18770 [cs.SE]. dirección: https://arxiv.org/abs/2406.18770

[8] D. Vungarala, S. Alam, A. Ghosh y S. Angizi. «SPICEPilot: Navigating SPICE Code Generation and Simulation with AI Guidance.» arXiv: 2410.20553 [cs.SE]. dirección: https://arxiv.org/abs/2410.20553

[9] Ngspice Development Team, *Ngspice User's Manual*, Version 44. dirección: https://ngspice.sourceforge.io/docs/ngspice-manual.pdf

[10] LangChain. «LangGraph: Building stateful, multi-actor applications with LLMs. »dirección: https://github.com/langchain-ai/langgraph

[11] F. Salvaire, *PySpice: Simulate electronic circuit using Python and the Ngspice/Xyce simulators*, 2021. dirección: https://github.com/PySpice-org/PySpice

[12] A. Vaswani et al., «Attention Is All You Need,» en *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[13] T. B. Brown et al., «Language Models Are Few-Shot Learners,» en *Advances in Neural Information Processing Systems (NeurIPS)*, verificar lista completa de autores, 2020.

[14] Z. Ji et al., «Survey of Hallucination in Natural Language Generation,» *ACM Computing Surveys*, vol. 55, n.º 12, 2023, verificar volumen/número y páginas.

[15] M. Chen et al. «Evaluating Large Language Models Trained on Code.» verificar eprint y lista de autores. arXiv: 2107.03374 [cs.LG]. dirección: https://arxiv.org/abs/2107.03374

[16] R. S. Pressman, *Ingeniería del Software: Un Enfoque Práctico*, 7.ª ed. McGraw-Hill, 2010.

[17] R. T. Fielding, «Architectural Styles and the Design of Network-based Software Architectures,» Tesis doct., University of California, Irvine, 2000.

[18] A. S. Sedra y K. C. Smith, *Microelectronic Circuits*, 7.ª ed. Nueva York, EE. UU.: Oxford University Press, 2015, confirmar con Diego el libro de texto de referencia del dominio, ISBN: 978-0-19-933913-6.

[19] SkyWater Technology, *SkyWater SKY130 PDK*, 2023. dirección: https://skywater-pdk.readthedocs.io/en/main/index.html

[20] ISO/IEC, *ISO/IEC 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation*, International Organization for Standardization, 2011.

[21] W. Stallings y L. Brown, *Computer Security: Principles and Practice*, 4.ª ed. Pearson, 2018, opcional; ancla del principio CID. Alternativa: una publicación de NIST, ISBN: 978-0-13-479410-5.

[22] Better Auth. «Better Auth Documentation.» verificar URL y año contra la versión que uses. dirección: https://www.better-auth.com/

[23] OWASP Foundation. «OWASP Top 10: The Ten Most Critical Web Application Security Risks.» verificar año de la edición que cites; 2021 es la vigente. dirección: https://owasp.org/www-project-top-ten/

Apéndice A

Diagramas de casos de uso

Los siguientes diagramas complementan el modelo de casos de uso presentado en la sección 2.2.2. Cada uno detalla un subconjunto de interacciones del sistema.

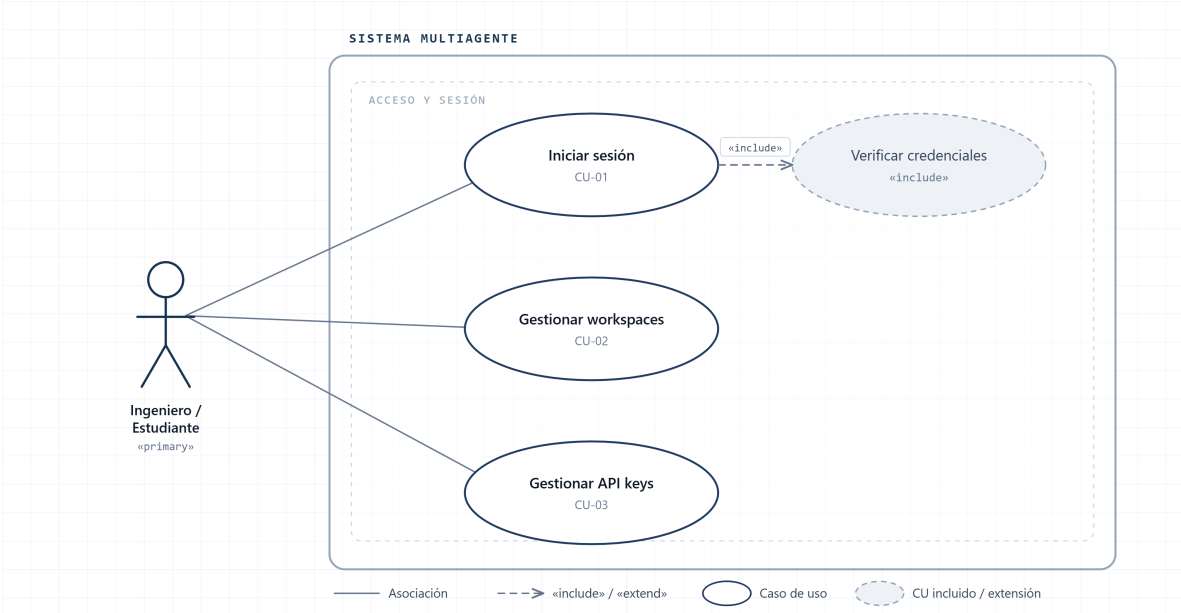


Figura A.1. Diagrama de casos de uso: acceso y gestión de sesiones.

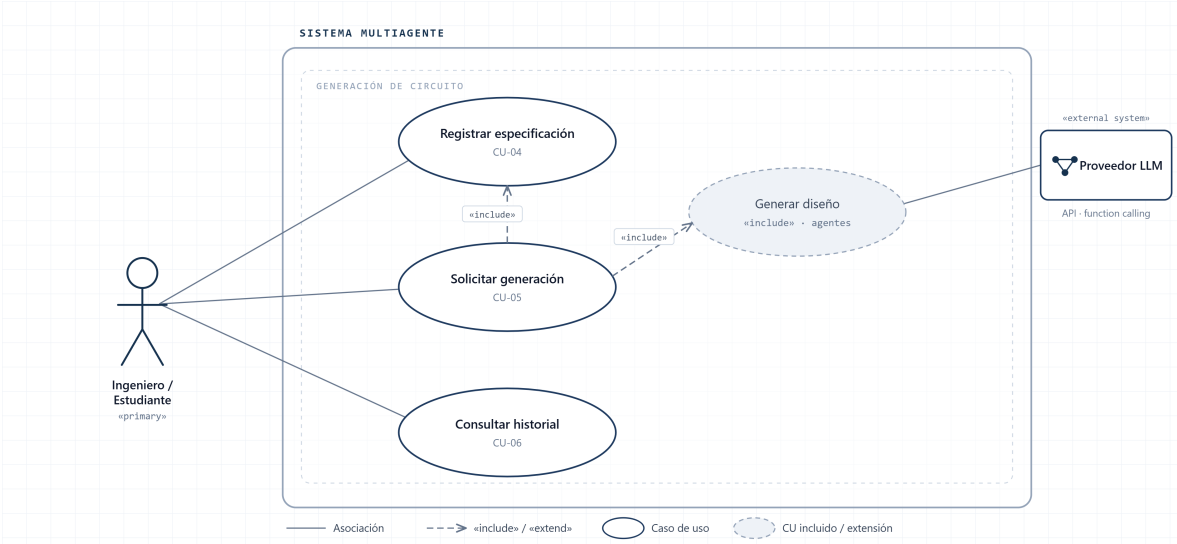


Figura A.2. Diagrama de casos de uso: generación de circuito.

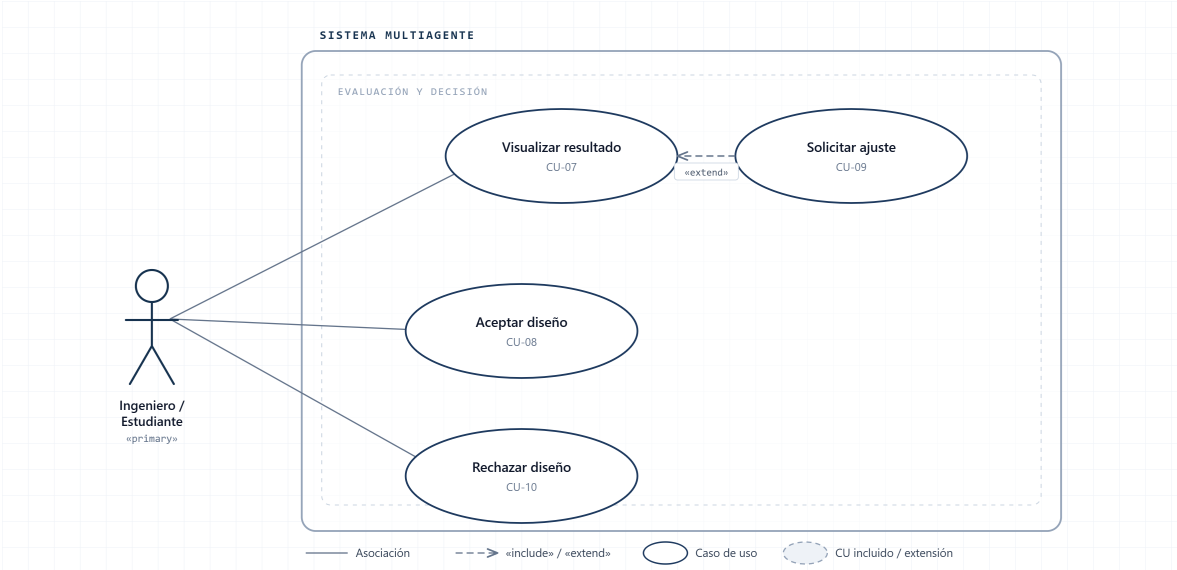


Figura A.3. Diagrama de casos de uso: decisión del agente curador.